Assignment 3

grep and regex

Introduction to UNIX Lab (NCSC102)

Instructions:

- **Once completed, show your code to the respective TAs.**
- **You may ask the TAs for help if you're encountering syntax errors or having trouble understanding the questions.**

Setup — create fresh lab data

Run the block below in your terminal to create a clean workspace and new files:

```
mkdir -p labdata/{src,system,archive}
cd labdata

# 1) Logs with varied severities (changed data)
cat > syslog.txt <<'EOF'
ALERT: fan speed drop
notice: backup completed
critical: voltage spike
Info: daemon started
DEBUG: packet flushed
alert: door opened
Warning: cache pressure
EOF

# 2) Event strings for word vs substring
cat > events.txt <<'EOF'
INFO: job queued
notevent-tag
Event horizon reached
info: job finished
reventful day
EOF

# 3) Tokens for line-count vs occurrence-count
cat > tokens2.txt <<'EOF'
nova beta nova
gamma NOVA Beta
beta-nova
EOF

# 4) Student-like IDs (new pattern)
cat > student_ids.txt <<'EOF'
24JE-0001
25JE-0668
25MT-0864
```

```
25je-867
AI23-099
EOF

# 5) Phones (simple validation)
cat > phones2.txt <<'EOF'
9123456780
+91-91234-56780
5987654321
70123 45678
9234567890
EOF

# 6) Usernames (mixture of valid/invalid)
cat > usernames.txt <<'EOF'
aaron99
Zed
mia_02
99bravo
xX_nova_Xx
neo07
EOF

# 7) Mixed ”loggy” lines for extraction
cat > mixed.log <<'EOF'
ALERT system: disk at 95%
notice: rotation complete
CRITICAL: kernel panic
debug: verbose off
Info: done
EOF

# 8) Small recursive tree for -r, --include/--exclude-dir
cat > src/service.log <<'EOF'
retry timeout reached
socket timeout on port 443
EOF

cat > src/readme.md <<'EOF'
Service docs. Do not log secrets.
EOF

cat > system/daemon.log <<'EOF'
TIMEOUT while connecting to db
EOF

cat > system/audit.txt <<'EOF'
user alice logged in
EOF

mkdir -p archive
cat > archive/old.log <<'EOF'
timed out after 30s
EOF
```

Part A — `grep` basics

A1: Case-insensitive match
Show all lines (with numbers) in **`syslog.txt`** that mention **`alert`** regardless of case.
Hint: combine **`-i`** and **`-n`**.

A2: Invert + anchor
From **`events.txt`**, print lines *not* starting with **`info`** (any case).
Hint: use **`-v`** with **`-i`** and **`^`**.

A3: Count
How many lines in **`events.txt`** contain the substring **`job`** (case-insensitive)?
Hint: **`-c -i`**.

A4: Whole word vs substring
In **`events.txt`**, compare a substring search for **`event`** vs. a whole-word search for **`event`**. Explain why **`notevent-tag`** and **`reventful`** behave differently.

A5: Only the match
From **`mixed.log`**, extract just the severity token at the *start* of each line where it is one of **`ALERT|notice|CRITICAL|debug|Info`**.
Hint: **`-Eo`** with **`^(ALERT`**notice|CRITICAL|debug|Info)|.

Part B — Regex practice (30 min)

B1: Validate ID format
In **`student_ids.txt`**, match IDs that are exactly: three uppercase letters, two digits, a hyphen, three digits. Print matching lines with their numbers.
Example idea: **`^[A-Z]{3}[0-9]{2}-[0-9]{3}$`**

B2: Simple phone check
In **`phones2.txt`**, match *only* lines that are 10 digits starting with 6–9 (no spaces/symbols).
Example idea: **`^[6-9][0-9]{9}$`**

B3: Username screening
From **`usernames.txt`**, print lines that: start with a letter; can contain letters/underscores in the middle; may end with exactly two digits (optional); and are at least 3 characters total. Use a single regex and show line numbers.

B4: Anchors & groups
From **`syslog.txt`**, print lines that *start* with either **`ALERT`**, **`Warning`**, or **`Info`** (respect case as written).
Hint: use **`^`** plus grouping with **`|`**.

Part C — Line vs occurrence counting

C1: In **`tokens2.txt`**, compute:

(A) number of *matching lines* for **`nova`** (case-insensitive),

(B) total *occurrences* of **`nova`** (case-insensitive), counting multiple per line.

Deliver both numbers and the commands you used.

Part D — Recursive search with filters (25 min)

D1: Find "timeout" anywhere
Search *recursively* under `.` for the word `timeout` (case-insensitive) in any file. Show file paths and line numbers.

D2: Restrict by extension
Repeat D1 but only within `*.log` files using `--include`.

D3: Exclude a directory
Repeat D1 but *exclude* the `archive/` directory with `--exclude-dir`.

D4: Synthesize
Which commands from D1–D3 will (a) report matches in `src/service.log`, (b) `system/daemon.log`, (c) `archive/old.log`? Explain briefly.

Stretch (optional)

S1: IPv4-ish tokens
Using the crude IPv4 pattern `([0-9]{1,3}\.){3}[0-9]{1,3}`, create a new file `net.txt` (some lines with IP-looking tokens, some not) and extract only the IPv4-ish tokens with `-Eo`.

S2: Glob vs regex quick check
Using shell globs (not regex), list all `*.log` files anywhere under `labdata` (recursive). Then confirm with a `grep -r` that each of those files either has or lacks the string `timeout`. Consider `**/*.log` with `globstar` or `find` as a fallback.

Deliverables

Create a `solutions.txt` that, for each task number, includes:

- the exact command you ran,
- the resulting output (redirected),

Instructor note: Tasks map directly to core regex concepts (anchors, classes, quantifiers, groups, alternation) and essential `grep` flags (`-i`, `-v`, `-c`, `-n`, `-w`, `-E`, `-o`, `-r`, `--include`, `--exclude-dir`).